

Copulas - Modellierung von Risikoabhängigkeiten mit Anwendung in der Portfoliooptimierung

Ilyas Bezzani

Lade Apple und Amazon tägliche Log-Renditen

```
getSymbols(c("AAPL", "AMZN"), from="2021-01-01", to="2026-01-01")
```

```
## [1] "AAPL" "AMZN"
```

```
aapl_ret = dailyReturn(Cl(AAPL), type = "log")
amzn_ret = dailyReturn(Cl(AMZN), type = "log")

data = na.omit(merge(aapl_ret, amzn_ret))

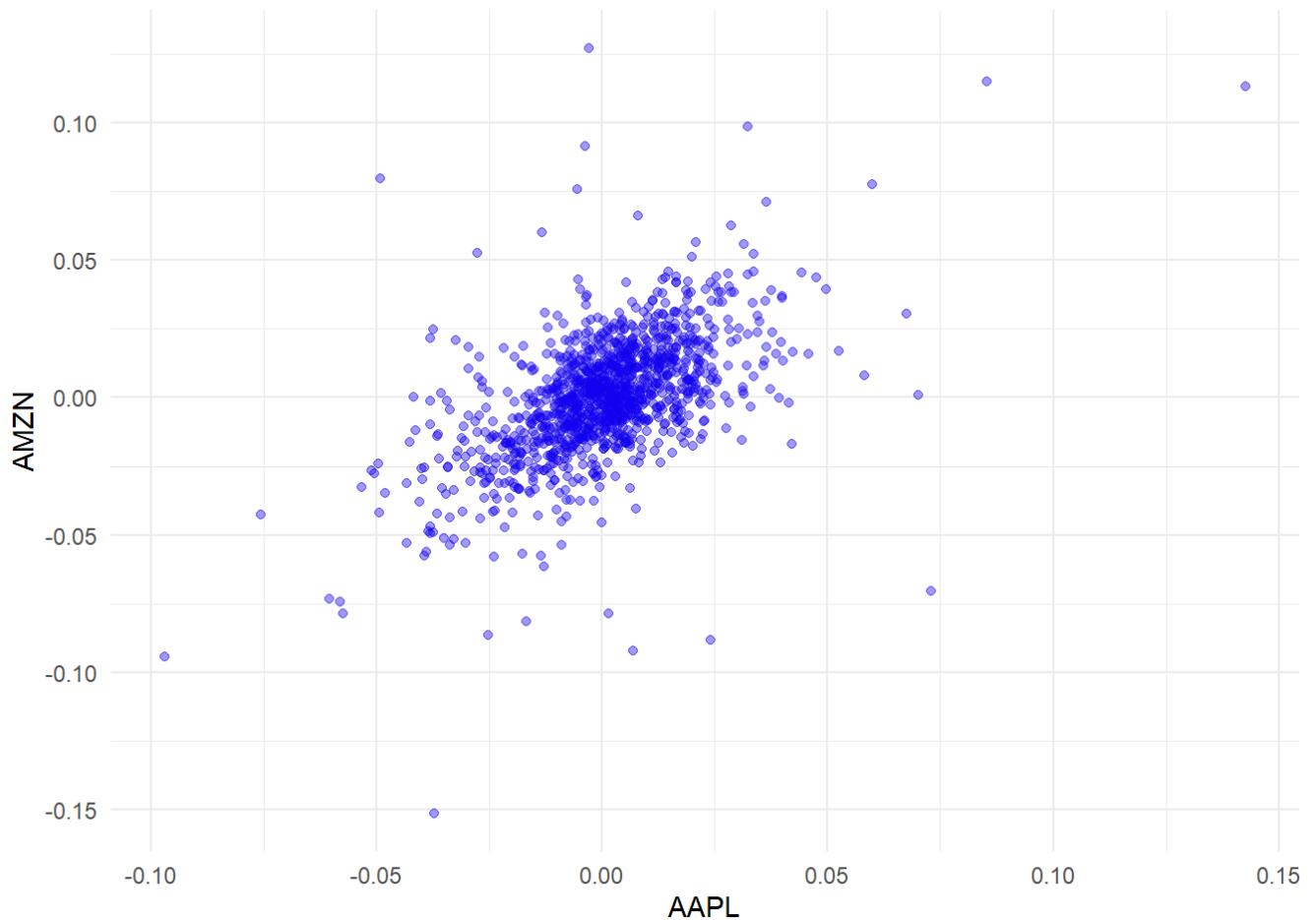
colnames(data) = c("AAPL", "AMZN")

length(data)
```

```
## [1] 2510
```

Scatterplot der Logrenditen, Visualisierung der Abhängigkeit

```
ggplot(as.data.frame(data),
       aes(x=AAPL, y= AMZN)) +
  geom_point(alpha=0.4, col="#1100ee")+
  theme_minimal()
```



Wir fitten jetzt passende Verteilungen auf die Marginale, also die Log-Renditen von AAPL und AMZN. Dazu wählen wir Gauß und t Verteilungen, fitten die Parameter mittels MLE und zeigen die verteilungen in den folgenden Histogrammen an.

```
# Location-Scale t-Verteilung definieren
dt_ls = function(x, df, mean, sd) {
  dt((x - mean) / sd, df = df) / sd
}

# t-Verteilungen fitten
fit_t_aapl = fitdist(
  data$AAPL,
  dt_ls,
  start = list(df = 5, mean = 0, sd = 0.02)
)
```

```
## Warning in fitdist(data$AAPL, dt_ls, start = list(df = 5, mean = 0, sd =
## 0.02)): The pt_ls function must be defined
```

```
fit_t_amzn = fitdist(
  data$AMZN,
  dt_ls,
  start = list(df = 5, mean = 0, sd = 0.02)
)
```

```
## Warning in fitdist(data$AMZN, dt_ls, start = list(df = 5, mean = 0, sd =  
## 0.02)): The pt_ls function must be defined
```

```

# Normalverteilungs-Parameter
mu_aapl = mean(data$AAPL)
sd_aapl = sd(data$AAPL)

mu_amzn = mean(data$AMZN)
sd_amzn = sd(data$AMZN)

# t-Verteilungs-Parameter Apple
df_t_aapl = fit_t_aapl$estimate["df"]
mean_t_aapl = fit_t_aapl$estimate["mean"]
sd_t_aapl = fit_t_aapl$estimate["sd"]

# t-Verteilungs-Parameter Amazon
df_t_amzn = fit_t_amzn$estimate["df"]
mean_t_amzn = fit_t_amzn$estimate["mean"]
sd_t_amzn = fit_t_amzn$estimate["sd"]

# Apple Histogramm
hist_aapl = ggplot(as.data.frame(data),
                    aes(x = AAPL)) +

  geom_histogram(
    aes(y = after_stat(density)),
    bins = 50,
    alpha = 0.5,
    fill = "lightblue",
    color = "black"
  ) +

  # Normalverteilung
  stat_function(
    fun = dnorm,
    args = list(
      mean = mu_aapl,
      sd = sd_aapl
    ),
    color = "red",
    linewidth = 1
  ) +

  # t-Verteilung
  stat_function(
    fun = function(x)
      dt((x - mean_t_aapl) / sd_t_aapl,
        df = df_t_aapl) / sd_t_aapl,
    color = "blue",
    linetype = "dashed",
    linewidth = 1
  ) +

  ggtitle("AAPL Log Returns") +
  theme_minimal()

# Amazon Histogramm

```

```

hist_amzn = ggplot(as.data.frame(data),
                    aes(x = AMZN)) +

  geom_histogram(
    aes(y = after_stat(density)),
    bins = 50,
    alpha = 0.5,
    fill = "lightgreen",
    color = "black"
  ) +

  # Normalverteilung
  stat_function(
    fun = dnorm,
    args = list(
      mean = mu_amzn,
      sd = sd_amzn
    ),
    color = "red",
    linewidth = 1
  ) +

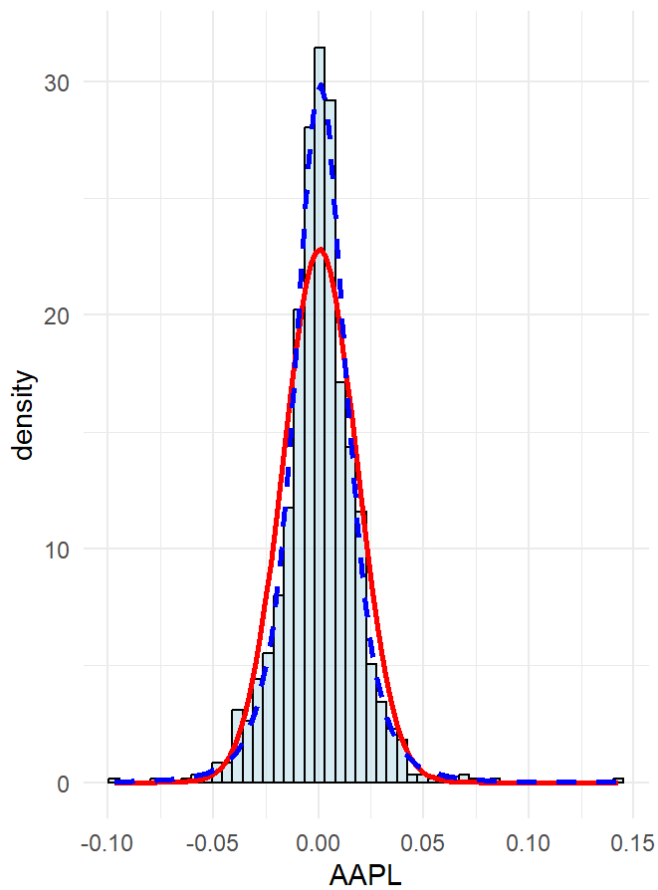
  # t-Verteilung
  stat_function(
    fun = function(x)
      dt((x - mean_t_amzn) / sd_t_amzn,
        df = df_t_amzn) / sd_t_amzn,
    color = "blue",
    linetype = "dashed",
    linewidth = 1
  ) +

  ggtitle("AMZN Log Returns") +
  theme_minimal()

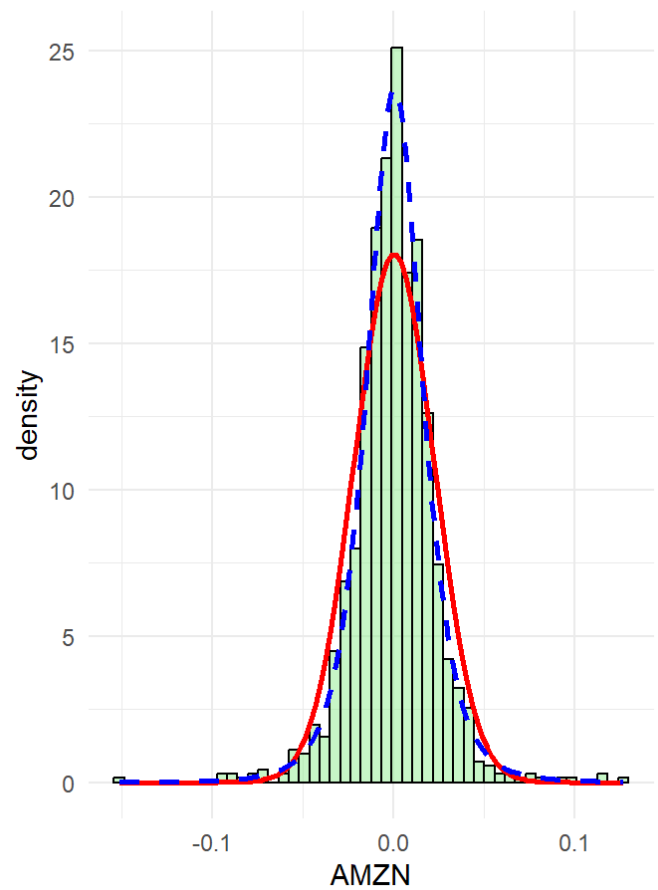
# Beide Histogramme anzeigen
grid.arrange(hist_aapl, hist_amzn, ncol = 2)

```

AAPL Log Returns



AMZN Log Returns



```
df_t_amzn
```

```
##      df
## 4.034363
```

```
df_t_aapl
```

```
##      df
## 3.941932
```

```
df_t_amzn
```

```
##      df
## 4.034363
```

Nun transformieren wir die historischen Daten mittels der gefitteten t-verteiling auf $\text{uniform}(0,1)$.

```

u1 = pt(
  (data$AAPL - mean_t_aapl) / sd_t_aapl,
  df=df_t_aapl
)
u2 = pt(
  (data$AMZN - mean_t_amzn) / sd_t_amzn,
  df=df_t_amzn
)

U = cbind(u1,u2)
U = as.matrix(U)

head(U)

```

```

##              AAPL      AMZN
## 2021-01-04 0.47419240 0.48739395
## 2021-01-05 0.79278527 0.70893949
## 2021-01-06 0.02480798 0.08871743
## 2021-01-07 0.96977316 0.66039891
## 2021-01-08 0.71433498 0.63736099
## 2021-01-11 0.06233000 0.11501702

```

Nun können wir versuchen, eine geeignete Copula zu finden, die die Abhängigkeitsstruktur der Log Renditen beschreibt. Wir versuchen Gauss, t, Frank und Clayton Copula, fitten ihre Parameter mittels MLE und vergleichen die LogLikelihood Werte, sowie visuelle Unterschiede.

```

gauss_cop = normalCopula(dim=2)
fit_gauss = fitCopula(
  gauss_cop,
  U,
  method="ml"
)

t_cop = tCopula(dim=2)
fit_t = fitCopula(
  t_cop,
  U,
  method="ml"
)

clayton_cop = claytonCopula(dim=2)
fit_clayton = fitCopula(
  clayton_cop,
  U,
  method="ml"
)

frank_cop = frankCopula(dim=2)
fit_frank = fitCopula(
  frank_cop,
  U,
  method="ml"
)

summary(fit_gauss)

```

```

## Call: fitCopula(gauss_cop, U, method = "ml")
## Fit based on "maximum likelihood" and 1255 2-dimensional observations.
## Normal copula, dim. d = 2
##      Estimate Std. Error
## rho.1  0.5743      0.016
## The maximized loglikelihood is 251.1
## Optimization converged
## Number of loglikelihood evaluations:
## function gradient
##           8           8

```

```
summary(fit_t)
```



```
## Call: fitCopula(t_cop, U, method = "ml")
## Fit based on "maximum likelihood" and 1255 2-dimensional observations.
## t-copula, dim. d = 2
##      Estimate Std. Error
## rho.1    0.5918      0.020
## df       3.9905      0.573
## The maximized loglikelihood is 300.6
## Optimization converged
## Number of loglikelihood evaluations:
## function gradient
##      14      14
```

```
summary(fit_clayton)
```

```
## Call: fitCopula(clayton_cop, U, method = "ml")
## Fit based on "maximum likelihood" and 1255 2-dimensional observations.
## Clayton copula, dim. d = 2
##      Estimate Std. Error
## alpha    1.334      0.063
## The maximized loglikelihood is 230.5
## Optimization converged
## Number of loglikelihood evaluations:
## function gradient
##      3      3
```

```
summary(fit_frank)
```

```
## Call: fitCopula(frank_cop, U, method = "ml")
## Fit based on "maximum likelihood" and 1255 2-dimensional observations.
## Frank copula, dim. d = 2
##      Estimate Std. Error
## alpha    4.299      0.197
## The maximized loglikelihood is 247.9
## Optimization converged
## Number of loglikelihood evaluations:
## function gradient
##      5      5
```

Die t Copula hat mit 271.8 den höchsten loglikelihood Wert mit Parametern $df=3.751$ und $\rho=0.56$. Dennoch wollen wir den Unterschied der Copulas graphisch vergleichen. Dazu ziehen wir zunächst uniforme Samples aus den Copulas

```
n = nrow(U)

Ug = rCopula(n, fit_gauss@copula)
Ut = rCopula(n, fit_t@copula)
Uc = rCopula(n, fit_clayton@copula)
Uf = rCopula(n, fit_frank@copula)
```

Diese werden wieder auf ihre t-marginals zurücktransformiert.

```

inv_aapl = function(u) {
  mean_t_aapl + sd_t_aapl*qt(u, df=df_t_aapl)
}
inv_amzn = function(u) {
  mean_t_amzn + sd_t_amzn*qt(u, df=df_t_amzn)
}

```

Nun können wir aus unserer modellierten gemeinsamen Verteilungsfunktion (die wegen Sklar's Theorem durch Copula und Marginals definiert ist) sampeln.

```

sample_gauss = data.frame(
  AAPL = inv_aapl(Ug[,1]),
  AMZN = inv_amzn(Ug[,2])
)
sample_t = data.frame(
  AAPL = inv_aapl(Ut[,1]),
  AMZN = inv_amzn(Ut[,2])
)
sample_clayton = data.frame(
  AAPL = inv_aapl(Uc[,1]),
  AMZN = inv_amzn(Uc[,2])
)
sample_frank = data.frame(
  AAPL = inv_aapl(Uf[,1]),
  AMZN = inv_amzn(Uf[,2])
)

```

Jetzt haben wir die Daten, um die gesampelten Scatterplots zu vergleichen.

```

x_lim = range(min(data$AAPL), max(data$AAPL))
y_lim = range(min(data$AMZN), max(data$AMZN))

# y_lim = range(
#   c(sample_gauss$AMZN,
#     sample_t$AMZN,
#     sample_clayton$AMZN,
#     sample_frank$AMZN)
# )

p1 = ggplot(sample_gauss,
             aes(AAPL, AMZN)) +
  geom_point(alpha=0.4, col="#1100ee") +
  ggtitle("Gauß Copula")+
  coord_cartesian(
    xlim = x_lim,
    ylim = y_lim
  )+
  theme_minimal()

p2 = ggplot(sample_t,
             aes(AAPL, AMZN)) +
  geom_point(alpha=0.4, col="#1100ee") +
  ggtitle("t Copula")+
  coord_cartesian(
    xlim = x_lim,
    ylim = y_lim
  )+
  theme_minimal()

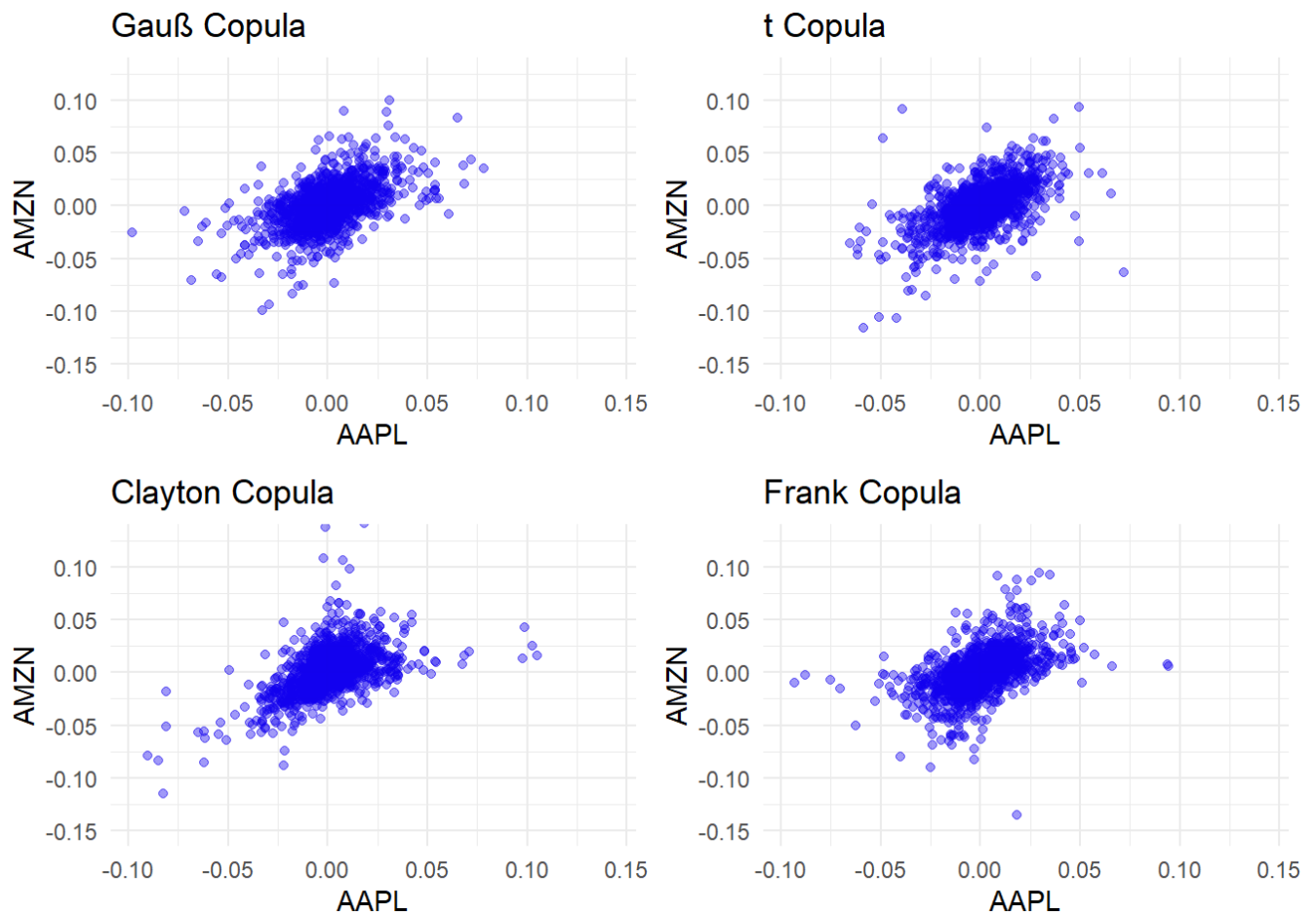
p3 = ggplot(sample_clayton,
             aes(AAPL, AMZN)) +
  geom_point(alpha=0.4, col="#1100ee") +
  ggtitle("Clayton Copula")+
  coord_cartesian(
    xlim = x_lim,
    ylim = y_lim
  )+
  theme_minimal()

p4 = ggplot(sample_frank,
             aes(AAPL, AMZN)) +
  geom_point(alpha=0.4, col="#1100ee") +
  ggtitle("Frank Copula")+
  coord_cartesian(
    xlim = x_lim,
    ylim = y_lim
  )+
  theme_minimal()

grid.arrange(
  p1, p2,
  p3, p4,
  ncol=2

```

)



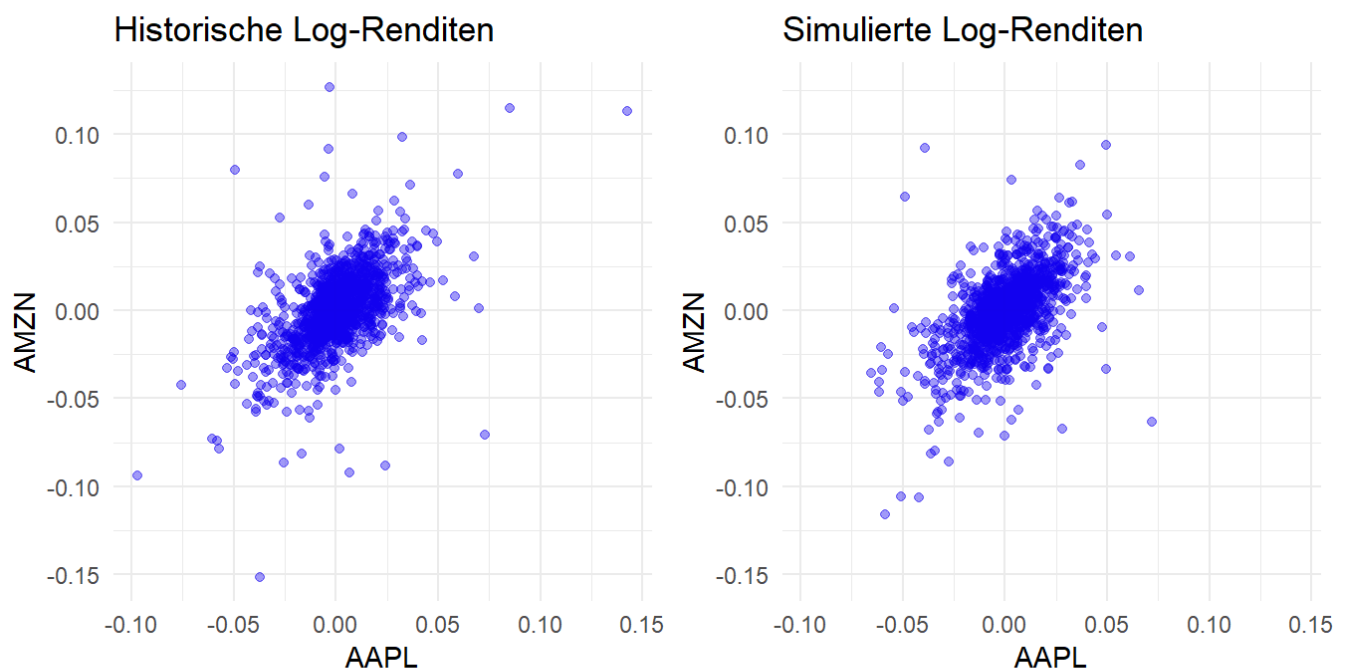
Vergleicht man das mit dem Scatterplot der historischen Daten, dann ist die t Copula auch visuell der beste Fit. Sie ist gekennzeichnet durch Tail Dependence (Extremereignisse treten oft gemeinsam auf).

```
pp1 = ggplot(as.data.frame(data),
             aes(x=AAPL, y= AMZN)) +
  geom_point(alpha=0.4, col="#1100ee")+
  ggtitle("Historische Log-Renditen") +
  theme_minimal()

pp2 = ggplot(sample_t,
             aes(AAPL, AMZN)) +
  geom_point(alpha=0.4, col="#1100ee") +
  ggtitle("Simulierte Log-Renditen")+
  coord_cartesian(
    xlim = x_lim,
    ylim = y_lim
  )+
  theme_minimal()

pp1 = pp1 + theme(aspect.ratio = 1)
pp2 = pp2 + theme(aspect.ratio = 1)

grid.arrange(pp1,pp2, ncol=2)
```



Die modellierte Verteilungsfunktion kann jetzt genutzt werden, um ein Portfolio aus AAPL und AMZN zu bauen, dass den CVaR minimiert.

Wir sampeln mittels einer Monte-Carlo-Simulation verschiedene Log-Renditen Paare und berechnen aus diesen Werten den (empirischen) VaR und CVaR.

```
n_sim = 100000

U_sim = rCopula(
  n_sim,
  fit_t@copula
)

sim_aapl = inv_aapl(U_sim[,1])
sim_amzn = inv_amzn(U_sim[,2])

sim_returns = cbind(sim_aapl, sim_amzn)

colnames(sim_returns) = c("AAPL", "AMZN")
```

Die CVaR Funktion der Verluste ist definiert durch

```

cvar_function = function(port_return_losses, alpha=0.95) {
  #alpha-Quantil, an dem Verluste VaR übersteigen (5% höchste Verluste)
  #VaR
  var_alpha = quantile(
    port_return_losses,
    probs = alpha
  )

  #Durchschnitt der Verluste über VaR
  cvar = mean(port_return_losses[port_return_losses >= var_alpha])

  return(cvar)
}

```

Das Gesamtportfolio (bzw. seine LogRenditen) approximieren wir durch $R_P = wR_{AAPL} + (1-w)R_{AMZN}$ für w in $[0,1]$. Es genügt also in einer Variable zu minimieren.

```

# Zielfunktion
objective_function = function(w_aapl) {
  # Nebenbedingungen
  if(w_aapl < 0 || w_aapl > 1){
    return(1e6)
  }

  w_amzn = 1- w_aapl

  pf_returns = w_aapl*sim_returns[,1]+w_amzn*sim_returns[,2]
  pf_losses = -pf_returns

  cvar = cvar_function(pf_losses, alpha=0.95)

  return(cvar)
}

```

Numerische Optimierung liefert nun

```

opt_result = optimize(
  f=objective_function,
  interval=c(0,1)
)

opt_result

```

```

## $minimum
## [1] 0.7501124
##
## $objective
## [1] 0.03792239

```

Die optimalen Portfolio-Gewichte sind also

```
w_aapl_opt = opt_result$minimum
w_amzn_opt = 1-opt_result$minimum

cat("Optimales Gewicht AAPL:", round(w_aapl_opt,4), "\n")
```

```
## Optimales Gewicht AAPL: 0.7501
```

```
cat("Optimales Gewicht AMZN:", round(w_amzn_opt,4), "\n")
```

```
## Optimales Gewicht AMZN: 0.2499
```

Eigenschaften des optimalen Portfolios sind

```
# Optimale Portfolio Returns
optimal_returns =
  w_aapl_opt * sim_returns[,1] +
  w_amzn_opt * sim_returns[,2]

# Kennzahlen
portfolio_mean = mean(optimal_returns)

portfolio_sd = sd(optimal_returns)

portfolio_var = quantile(
  optimal_returns,
  0.05
)

portfolio_cvar = mean(optimal_returns[optimal_returns<=portfolio_var])

cat("Erwartete Rendite:",
    round(portfolio_mean, 5), "\n")
```

```
## Erwartete Rendite: 9e-04
```

```
cat("Volatilität:",
    round(portfolio_sd, 5), "\n")
```

```
## Volatilität: 0.01714
```

```
cat("VaR(5%):",
    round(portfolio_var, 5), "\n")
```

```
## VaR(5%): -0.02493
```

```
cat("CVaR(5%):",
    round(portfolio_cvar, 5), "\n")
```

```
## CVaR(5%): -0.03792
```

```
ggplot(  
  data.frame(LogRenditen = optimal_returns),  
  aes(x = LogRenditen)  
) +  
  
  geom_histogram(  
    aes(y = after_stat(density)),  
    bins = 100,  
    fill = "blue",  
    color = "black",  
    alpha = 0.7  
  ) +  
  
  geom_vline(  
    xintercept = portfolio_var,  
    color = "red",  
    linewidth = 1.2,  
    linetype = "dashed"  
  ) +  
  
  annotate(  
    "label",  
    x = portfolio_var-0.1,  
    y = 20,  
    label = paste0("VaR = ", round(portfolio_var, 4)),  
    fill = "white",  
    color = "red"  
  ) +  
  annotate(  
    "label",  
    x = portfolio_var-0.15,  
    y = 2,  
    label = paste0("CVaR = ", round(portfolio_cvar, 4)),  
    fill = "white",  
    color = "red"  
  ) +  
  theme_minimal()
```